Kaaval Assurance

Routers predict. Kaaval verifies.

An inference assurance plane for open-weight models on AMD compute.

Verify, not predict. Receipts, not promises.

The Problem: Accountability Gap

The Accountability Gap: If an AI agent makes the wrong transaction, the enterprise still owns the customer outcome and liability.

The Fluent Failure Mode: Unsafe AI answers are fluent and grammatically correct—but factually or legally wrong. Standard guardrails check syntax; they cannot verify business logic.

The Business Risk: As agents handle refunds, claims, and quotes, quality drift translates directly into financial exposure.

The Solution: Kaaval Product

1. **Local First:** Run task on cheap, local open-weight models (Gemma) on owned hardware.
2. **Real-time Gate:** Intercept response and test against a strict code contract *before* it ships.
3. **Fail-Safe Escalation:** Automatically route to a remote backup tier only when local verification fails.
4. **Decision Receipts:** Store every transaction, latency, cost, and verifier check in an auditable database.

Three-Layer Assurance Architecture

Layer 1: Contract Conformance

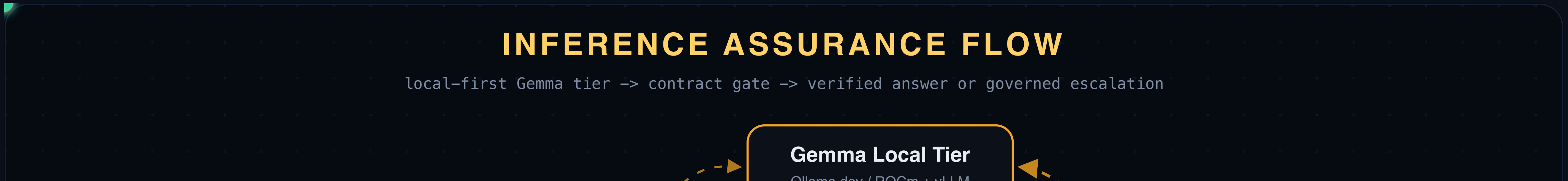
Structured JSON schema, enums, range checks, and grounding rules. Deterministic, code-only checks (no LLM judging LLM).

Layer 2: Adaptive Drift Routing

EWMA failure tracking. Automatically tightens routing thresholds when local quality degrades.

Layer 3: Sampled Offline Audit

Calibration-gated adversarial auditing. Critic model challenges a 10% sample of accepted answers.



The Telemetry Truth (Evidence)

No claim without a stored field. No field without a source tag.

Metric	Captured Value	Source Tag
Local Conformance Rate	`100.0%`	`measured`
Final Verified Rate	`100.0%`	`measured`
Escalation Rate	`0.0%`	`measured`
Latency p50 / p95	`324.6 / 479.6 ms`	`measured`
Inference Cost	`$0.0000 (Local-First)`	`measured`
Active Run ID	`live-5be3acfa-amd-gemma-proof`	`measured`

AMD Proof & Hardware Stack

- **Hardware Stack:** AMD Radeon gfx1100 target, 47.98 GiB VRAM.
- **Software Stack:** vLLM served via ROCm 7.2 + PyTorch 2.9.
- **Measured Proof Bundle:** Locked to source commit `aa8b5b2` with SHA-256 integrity hashes (`SHA256SUMS-amd-aa8b5b2.txt`).

Market Positioning

Traditional Routers (Guess & Hope):

Focus on *predicting* model quality before generation. Useful for text completion, but fails when business contracts cannot tolerate errors.

Kaaval Assurance (Verify & Adapt):

Focus on *measuring* output conformance after generation. Perfect for policy-bound decisions, API spend governance, and auditable automation.